

Este segundo trabajo practico consistirá en proveer un nuevo servicio de persistencia utilizando Spring Data JPA en lugar de usar Hibernate directamente, como se hizo en el primer trabajo práctico.

Para lograr esto, deberá crearse una nueva implementación de la interfaz **ToursService** llamada **ToursServiceImpl**, tomando como base la implementación anterior, que deberá utilizar repositorios de Spring Data para resolver cualquier acceso a datos requerido.

En este caso se debe crear un repositorio por clase a persistir. Los repositorios implementados con Spring Data no son clases sino interfaces, y deberán extender `CrudRepository` definiendo los argumentos necesarios: tipo de entidad y tipo de id. Por ejemplo:

```
public interface RouteRepository extends CrudRepository<Route, Long>
```

Las consultas se pueden declarar utilizando query methods, los cuales serán implementados automáticamente por Spring Data (por ejemplo: `findBy<Propiedad>`), o mediante HQL utilizando la annotation `@Query` (notar que, dado que se trata de una interfaz, los métodos no pueden tener implementación).

Las clases de tests, `ToursApplicationTests` y `ToursQueryTests` validan, al igual que en la etapa previa, distintas implementaciones del servicio para que pueda probar libremente estas.

Para configurar SpringData se utiliza la clase `SpringDataConfiguration`, que deberán editar con sus credenciales de acceso a la BD.

La infraestructura del proyecto provista se encuentra en el mismo repositorio del TP previo, en la rama [tp2-springdata](#):

En concreto, se han modificado: la clase de configuración `AppConfig`, se ha reemplazado `HibernateConfiguration` con la clase de configuración `SpringDataConfiguration` y se han actualizado las dependencias necesarias en `pom.xml`; `DBInitializer` y por supuesto, los archivos de Testing.

Descripción del dominio

Se quiere generar una app de viajes turísticos para un emprendimiento local.

Una vez que una persona se ha registrado en el sistema de Turismo, puede elegir alguno de los varios recorridos diarios que la empresa ofrece y comprarlo.

Cada recorrido (Route) tiene su nombre, que es representativo del tramo que visita, el valor del mismo, el total en km del recorrido y pueden visualizarse cada una de las paradas (Stop) que componen el recorrido. Todos los recorridos se realizan una vez por día. Además, la aplicación registra comentarios y valoraciones (estrellas) de otros usuarios que han realizado el mismo recorrido.

Dada la cantidad de personas que solicitan los recorridos, la empresa ha tenido que poner un tope máximo de personas por recorrido. Esa cantidad no es general sino que se define para cada uno y se deben vender solo mientras el cupo lo permita.

Adicionalmente, se conoce una serie de choferes y guías turísticos que participan del recorrido.

A modo de ejemplo, podemos pensar en el recorrido "Tour tradicional de Bs. As." que cuenta con paradas en distintos lugares emblemáticos de la ciudad de Buenos Aires como Caminito en La Boca, San Telmo, Obelisco, lago de Palermo y Recoleta; o en "Tour del Delta" que incluye a San Isidro, el Delta y también al Obelisco.

Una vez que se ha registrado la compra de un usuario para un recorrido determinado (Purchase), se le habilita la posibilidad de agregar comentarios y puntaje al recorrido realizado (Review) de forma tal que luego puedan usarlo en promociones del emprendimiento y poder captar mayor público interesado en los mismos.

Se ha pensado que los usuarios (User) se identifiquen en la aplicación mediante usuario y contraseña, y otros datos que pueden resultar útiles para posteriores ofertas de tours que la empresa realice. Por eso se les solicitan datos personales y de contacto como el nombre, un número de celular, un email, edad y fecha de nacimiento.

Los choferes (DriverUser) y guías turísticos (TourGuideUser) también son usuarios del sistema, por lo que se registran los mismos datos que los usuarios, pero además cuentan con una serie de antecedentes que la empresa quiere mantener. El chofer, de trabajos anteriores; el guía turístico, sobre su formación en turismo. Cabe mencionar que los choferes y los guías no están afectados a un recorrido en particular, sino que pueden participar de varios según la demanda y necesidad de la empresa.

Durante todo el recorrido se ofrecen diversos servicios (Service) que un usuario puede contratar. De los servicios se conoce su nombre, su valor, una descripción y el proveedor que lo brinda. Por ejemplo, la confección de algún recuerdo (un mate pintado artesanalmente, una remera o una foto en un lugar determinado).

De cada proveedor (Supplier) se conocen los servicios que ofrece, la razón social y un número de habilitación provincial.

Cada ítem de servicio aparece en la compra junto con el recorrido y la cantidad solicitada del mismo (ItemService).

La compra final contendrá el recorrido elegido y todos los servicios solicitados a lo largo del mismo para que el usuario pueda tener control de lo abonado. Además, sobre cada compra se registra un código interno y la fecha.

Sección 1 - Introducción a Spring Data JPA

Esta sección introduce el framework Spring Data JPA, su lugar dentro del ecosistema Spring y su relación con la infraestructura de persistencia construida en la Práctica 1. El objetivo es entender qué agrega este framework, cómo lo hace internamente y qué partes del proyecto anterior se conservan sin cambios.

1.1 Spring Data JPA y su lugar en el ecosistema

1. ¿Qué es Spring Data JPA? ¿Qué problema resuelve respecto de usar Hibernate directamente? Describir dos situaciones del proyecto donde Spring Data JPA simplifica código que en la Práctica 1 requería implementación manual.

Spring Data JPA es una capa de abstracción que simplifica el acceso a datos usando repositorios, evitando tener que describir manualmente muchas operaciones repetitivas.

Resuelve el tener que abrir/cerrar sesiones, escribir save, delete o muchas consultas simples a mano

Dos situaciones donde simplifica serían

1. Evitar código manual de save, get, delete
2. Tener que declarar métodos manualmente y sus queries
2. Spring Data JPA no es un ORM sino una capa de abstracción sobre el ORM. Explicar la diferencia: ¿qué hace Spring Data JPA? ¿Qué sigue haciendo Hibernate internamente?

Spring Data JPA es una capa de abstracción sobre JPA/Hibernate que se encarga de simplificar el acceso a datos mediante repositorios, query methods, paginación y generación automática de implementaciones de métodos

Hibernate sigue siendo quien hace internamente el trabajo de ORM

Spring Data JPA decide qué operación de repositorio ejecutar. Hibernate ejecuta realmente el mapeo y la persistencia

3. La siguiente tabla lista tareas relacionadas con la persistencia. Marcar con una X la columna de la tecnología que resuelve ese problema en la nueva implementación con Spring Data JPA:

Tarea	JDBC	Hibernate	Spring Data JPA
Abrir y cerrar la conexión a la base de datos	X		
Implementar save(), findById() y deleteById()			X
Gestionar el ciclo de vida de las entidades (@Entity)		X	
Derivar una consulta a partir del nombre del método			X
Manejar el pool de conexiones	X		
Propagar transacciones con @Transactional			X
Generar la implementación del repositorio en runtime			X
Mapear ResultSet a objetos Java		X	
Proveer soporte nativo de paginación (Pageable)			X

1.2 Configuración del proyecto

Migrar de Hibernate directo a Spring Data JPA implica cambios en la configuración del proyecto, pero no en el mapeo de entidades ni en la lógica de negocio. En la Práctica 1 la configuración de Hibernate se realizaba mediante una clase Java de configuración. En esta práctica se utilizará el archivo `application.properties` de Spring Boot, que centraliza toda esa configuración de forma declarativa.

4. Listar los cambios que deben realizarse en el proyecto de la Práctica 1 para incorporar Spring Data JPA.
 - Mantener el mapeo JPA/Hibernate de entidades, relaciones, herencia, cascadas y tipos de fetch
 - Reemplazar los repositorios manuales que usaban `SessionFactory` y `getCurrentSession` por interfaces Spring Data JPA
 - Crear un repositorio por entidad extendiendo `CrudRepository`
 - Eliminar la implementación manual de operaciones básicas
 - Reemplazar las consultas HQL por query methods o métodos con `@Query`
 - Reemplazar la configuración manual de Hibernate por `SpringDataConfiguration` y `application.properties`
 - Configurar `application.properties` con la conexión a la base, usuario, contraseña, driver, dialecto y `spring.jpa.hibernate.ddl-auto`
 - Actualizar `TourServiceImpl` para que use repositorio inyectados en lugar de acceder directamente a `Session`
 - Mantener las transacciones en la capa de servicio con `@Transactional`
5. En la Práctica 1 se configuraba Hibernate mediante una clase de configuración Java para obtener la `SessionFactory`. ¿Cómo se reemplaza esta configuración usando `application.properties` en Spring Boot?

Para reemplazar esa configuración usando `application.properties` se aprovecha que Spring Boot crea automáticamente el `DataSource`, el `EntityManagerFactory` y los repositorios `SpringData`

De esta forma, en lugar de obtener la session, se utilizan directamente los repositorios

6. Describir las propiedades más relevantes de Spring Data JPA que deben configurarse en `application.properties` para este proyecto. Incluir al menos: `datasource` (url, username, password, driver), dialecto de Hibernate, y la propiedad que controla si Hibernate debe crear, actualizar o validar el esquema. ¿Cuál de estos valores conviene usar durante el desarrollo?

Tip El mapeo de entidades (anotaciones `@Entity`, `@Table`, `@OneToMany`, `@ManyToMany`, estrategias de herencia, cascadas y `FetchType`) implementado en la Práctica 1 no debe modificarse. Spring Data JPA lo utiliza tal como está.

Las propiedades más relevantes para Spring Data JPA que deben configurarse son las relacionadas a la conexión de base de datos y con Hibernate/JPA

```
spring.datasource.url=jdbc:postgresql://localhost:5432/tours
spring.datasource.username=postgres
spring.datasource.password=postgres
spring.datasource.driver-class-name=org.postgresql.Driver

spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect

spring.jpa.hibernate.ddl-auto=update

spring.jpa.show-sql=true
```

Descripción

- spring.datasource.url
 - Define la URL de conexión a la db
 - spring.datasource.username
 - Usuario de la db
 - spring.datasource.password
 - Contraseña del usuario
 - spring.datasource.driver-class-name
 - Driver JDBC utilizado
 - spring.jpa.database-platform
 - Dialecto hibernate, que le indica cómo generar sql para el motor utilizado
 - spring.jpa.hibernate.ddl-auto
 - Controla qué hace hibernate con el esquema
 - create: crea el esquema de cero
 - create-drop: crea al iniciar y elimina al finalizar
 - update: actualiza el esquema manteniendo datos
 - validate: solo valida que coincida con el mapeo
 - none: no realiza cambios
- Durante el desarrollo conviene usar "update" porque permite modificar entidades y que Hibernate actualice automáticamente el esquema sin perder los datos existentes

1.3 La interfaz CrudRepository y la jerarquía de repositorios

El punto de entrada a Spring Data JPA es una jerarquía de interfaces de repositorio. Comprender cada nivel de esa jerarquía permite elegir la interfaz adecuada para cada caso de uso.

7. ¿Qué es CrudRepository? ¿De qué interfaz hereda y qué operaciones provee automáticamente?

CrudRepository es una interfaz de Spring Data JPA que provee automáticamente las operaciones básicas CRUD

Hereda de la interfaz `Repository<T, ID>` que actúa como interfaz marcador dentro de Spring Data

Al extender CrudRepository, Spring genera automáticamente métodos del CRUD como

```
<S extends T> S save(S entity);

Optional<T> findById(ID id);

Iterable<T> findAll();

void deleteById(ID id);

boolean existsById(ID id);

long count();
```

8. ¿Que agrega cada nivel de la jerarquía respecto del anterior? Describir brevemente las diferencias entre CrudRepository, PagingAndSortingRepository y JpaRepository, indicando que operaciones o capacidades incorpora cada uno

La jerarquía de repositorios de Spring Data agrega funcionalidades progresivamente

- CrudRepository
 - Operaciones CRUD básicas como save, findById, findAll, deleteById, existsById, count
 - PagingAndSortingRepository
 - Agrega soporte para paginación y ordenamiento mediante Pageable y Sort
 - JpaRepository
 - Agrega funcionalidades específicas de JPA, como flush, saveAndFlush, deleteAllInBatch, paginación avanzada y manejo más complejo de colecciones (List en lugar de Iterable)
9. Crear las interfaces de repositorio para las entidades del modelo extendiendo CrudRepository. Indicar los parámetros de tipo correctos (entidad e ID) para cada una: PurchaseRepository, RouteRepository, UserRepository, ServiceRepository, SupplierRepository y ReviewRepository.

10. ¿Cómo genera Spring Data JPA la implementación concreta de los repositorios en tiempo de ejecución? ¿Qué rol juega el proxy dinámico de Java en este mecanismo?

Spring Data JPA genera la implementación concreta de los repositorios en tiempo de ejecución

Cuando se define una interfaz como

```
public interface RouteRepository extends CrudRepository<Route, Long> {}
```

Spring detecta esa interfaz al iniciar la aplicación y crea un proxy dinámico que implementa sus métodos

Ese proxy se encarga de

- Ejecutar métodos ya existentes como save, findById, findAll, delete
 - Interpretar query methods
 - Ejecutar consultas anotadas con @Query
 - Delegar internamente en JPA/Hibernate para hacer la operación real sobre la base
11. ¿Que diferencia hay entre save() en Spring Data JPA y session.save() / session.merge() en Hibernate directo? ¿Cómo decide Spring Data JPA si debe hacer un INSERT o un UPDATE?

En Spring Data JPA, save() unifica operaciones que en Hibernate directo se hacían con métodos separados como session.save() y session.merge()

En Hibernate:

```
session.save(entity); // inserta una entidad nueva  
session.merge(entity); // actualiza o re-adjunta una entidad detached
```

En Spring Data JPA:

```
repository.save(entity);
```

save() sirve tanto para insertar como para actualizar. Spring Data JPA decide qué operación realizar según si la entidad es considerada nueva o no. Generalmente se determina observando el valor del id: si es nulo, es una entidad nueva y hace INSERT, si tiene un valor considera que la entidad ya existe y hace un UPDATE mediante merge

Sección 2 - Repositorios con Spring Data JPA

En esta sección se reemplazan los repositorios manuales de la Práctica 1 por interfaces Spring Data JPA. El objetivo es implementar la misma funcionalidad con menos código, explorando las distintas formas de definir consultas que ofrece el framework.

2.1 Migración de repositorios

12. Comparar el código de `PurchaseRepository` de la Práctica 1 (con `Session` de Hibernate) con el nuevo `PurchaseRepository` de Spring Data JPA. ¿Cuántas líneas de código se eliminaron? ¿Qué operaciones ya no es necesario implementar manualmente?

Práctica 1:

```
package unlp.info.bd2.repositories;

import org.hibernate.SessionFactory;
import unlp.info.bd2.model.Purchase;

import java.util.Date;
import java.util.List;
import java.util.Optional;

public class PurchaseRepository extends RepositoryBase<Purchase>
{
    public PurchaseRepository(SessionFactory sessionFactory) {
        super(sessionFactory);
    }

    public List<Purchase> getAllPurchasesOfUsername(String username){
        return this.getSession().createQuery("from Purchase where
user.username = :username", Purchase.class)
            .setParameter("username", username)
            .getResultList();
    }

    public long getCountOfPurchasesBetweenDates(Date start, Date end) {
        Long count = getSession()
            .createQuery("""
select count(p)
from Purchase p
where p.date between :start and :end
""", Long.class)
            .setParameter("start", start)
            .setParameter("end", end)
```

```

        .uniqueResult();
        return count != null ? count : 0L;
    }
}

```

Práctica 2:

```

package unlp.info.bd2.repositories;

import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.CrudRepository;
import unlp.info.bd2.model.Purchase;

import java.util.Date;
import java.util.List;

public interface PurchaseRepository extends CrudRepository<Purchase, Long>
{
    List<Purchase> findByUserUsername(String username);

    long countByDateBetween(Date start, Date end);
}

```

Se eliminaron bastantes líneas, ya no es necesario implementar el save, el findById, findAll, delete, obtención/apertura/cierre del session, consultas simples, conteos simples entre fechas

- En la Práctica 1, los repositorios gestionaban internamente la Session. ¿Que recibe ahora un servicio que necesita usar un repositorio Spring Data? ¿Cómo se declara esa dependencia? ¿Dónde queda ahora la lógica de apertura y cierre de sesiones?

Atención: Al migrar los repositorios, los servicios de la Práctica 1 que invocaban métodos de la Session directamente (session.save, session.get, etc.) deben actualizarse para usar los métodos del nuevo repositorio Spring Data. La lógica de negocio en sí no cambia.

La dependencia se declara mediante el constructor del servicio y Spring inyecta automáticamente la implementación generada del repositorio

La lógica de apertura y cierre de sesiones deja de estar en los repositorios y pasa a ser administrada automáticamente por Spring y JPA dentro del contexto transaccional definido con @Transactional

2.2 Query Methods

Spring Data JPA puede generar consultas automáticamente a partir del nombre del método definido en la interfaz del repositorio. Esta técnica se denomina Query Methods.

14. ¿Cómo funciona la generación de consultas por nombre de método en Spring Data JPA? Describir las palabras clave principales que puede interpretar (findBy, existsBy, countBy, deleteBy) y cómo se combinan con los atributos de la entidad.

Spring Data JPA puede generar consultas automáticamente a partir del nombre del método definido en el repositorio.

Spring analiza el nombre del método y construye internamente la consulta JPQL correspondiente utilizando los atributos de la entidad

La estructura general suele ser <operación>By<atributo><condición>

Principales palabras clave:

- findBy
 - Busca entidades

```
List<User> findByEmail(String email)
```
- existsBy
 - Verifica existencia

```
boolean existsByUsername(String username)
```
- countBy
 - Cuenta registros

```
long countByDateBetween(Date start, Date end)
```
- deleteBy
 - Elimina registros

```
void deleteByUsername(String username)
```

Estas palabras se combinan con atributos de la entidad y operadores como

- And
- Or
- Between
- LessThan
- GreaterThan
- Like
- Containing
- StartingWith
- EndingWith
- OrderBy

Por ejemplo:

```
List<Route> findByPriceLessThanOrderByPrice(float price)
```

- findBy -> búsqueda
- PriceLessThan -> price < ?
- OrderByName -> ordenado por nombre

15. ¿Cómo se pasan los parametros a un Query Method? ¿Cómo hace Spring Data JPA para asociar cada parámetro del método con la condición correspondiente en la consulta? ¿Qué ocurre si el orden de los parámetros no coincide con el orden de las condiciones en el nombre del método?

Los parámetros de un Query Method se pasan como parámetros normales del método

```
List<Route> findByPriceBetween(float min, float max);
```

Spring Data JPA asocia automáticamente cada parámetro con la condición correspondiente según el orden en el nombre del método y el orden de los parámetros del método

Por ejemplo

```
List<Service> findByNameAndPrice(String name, float price);
```

Spring interpreta name y price como primer y segundo parámetro respectivamente

Si el orden de los parámetros no coincide con el orden de las condiciones en el nombre del método, la consulta funcionará incorrectamente porque Spring asigna los valores posicionalmente

Por ejemplo

```
findByNameAndPrice(float price, String name)
```

Es incorrecto y generaría una consulta incorrecta o un error de tipos

16. Investigar y para cada uno de los siguientes requerimientos indicar si es posible resolverlo con un Query Method y escribir la firma del método correspondiente. Si no es posible, explicar por qué:

a) Buscar todas las Purchase de un usuario dado su username.

```
List<Purchase> findByUserUsername(String username);
```

b) Verificar si existe alguna Purchase para una Route dada.

```
boolean existsByRoute(Route route);
```

c) Contar cuantas Review tienen un rating mayor o igual a un valor dado.

```
long countByRatingGreaterThanOrEqualTo(int rating);
```

- d) Obtener todas las Route cuyo precio sea menor a un valor dado, ordenadas por nombre.

```
List<Route> findByPriceLessThanOrderByPrice(float price);
```

e) Buscar un User por su email.

```
Optional<User> findByEmail(String email);
```

- f) Obtener los top 3 Route con mayor cantidad de Purchase.

Como hay que agrupar, no se puede hacer con Query Methods

```
@Query("""
    select p.route
    from Purchase p
    group by p.route
    order by count(p) desc
""")
List<Route> getTop3RoutesWithMostPurchases(Pageable pageable);
```

17. Investigar cuales son las palabras clave (keywords) disponibles en Spring Data JPA para construir Query Methods. ¿Qué tipos de condiciones, comparaciones y operadores lógicos soporta?

Keywords comunes para Query Methods en Spring Data JPA

Tipo	Keywords
Búsqueda	findBy, readBy, getBy, queryBy, searchBy
Existencia	existsBy
Conteo	countBy
Borrado	deleteBy, removeBy
Comparación	LessThan, LessThanEqual, GreaterThan, GreaterThanEqual, Between
Igualdad	Is, Equals, Not, IsNot
Nulos	IsNull, IsNotNull
Texto	Like, NotLike, StartingWith, EndingWith, Containing

Tipo	Keywords
Booleanos	True , False
Colecciones	In , NotIn
Lógica	And , Or
Orden	OrderBy , Asc , Desc
Límite	Top , First
Duplicados	Distinct
Fechas	Before , After

Ejemplos

```

List<Route> findByPriceLessThanOrderByNameAsc(float price);

Optional<User> findByEmail(String email);

boolean existsByRoute(Route route);

long countByRatingGreaterThanOrEqual(int rating);

List<Service> findTop3ByPriceGreaterThanOrOrderByPriceDesc(float price);

List<User> findByEmailContainingIgnoreCase(String email);

```

Soportan consultas simples sobre atributos, comparaciones, texto, nulos, booleanos, ordenamiento y combinaciones con And/Or. Para agregaciones como sum, avg, count con group by, rankings complejos o subconsultas conviene usar @Query

- ¿Qué limitaciones tienen los Query Methods? Describir al menos tres casos del modelo donde esta técnica resulta insuficiente y es necesario otro enfoque. Describa los otros enfoques posibles para implementar dichas consultas.

Tip Los Query Methods son útiles para consultas simples sobre uno o dos atributos. Para consultas con JOINS complejos, agrupaciones o subconsultas, usar @Query es más claro y mantenible.

Los Query Methods tienen limitaciones porque sirven principalmente para consultas simples sobre atributos y relaciones directas. Entre las limitaciones están

- No manejan bien agregaciones complejas
 - No permiten expresar GROUP BY, COUNT, SUM, AVG
- Consultas con múltiples JOIN complejos

- Aunque se pueden usar propiedades anidadas, se vuelve limitado cuando hay varios JOINS o se necesitan condiciones complejas entre entidades
 - Consultas dinámicas complejas
 - No permiten armar queries condicionales del tipo
 - "Filtrar por precio si viene, por fecha si viene, etc."
 - El nombre del método no puede adaptarse dinámicamente
 - Consultas muy largas o difíciles de leer
 - Se vuelven poco mantenibles
 - Subconsultas o lógica avanzada
 - No soportan subqueries
 - No permiten lógica compleja dentro de la consulta
- Tres casos del modelo donde la técnica resulta insuficiente
- Top 3 rutas con más compras
 - Filtrado dinámico de Purchase (por usuario, fecha o precio opcionalmente)
 - Buscar Route con condiciones sobre múltiples relaciones

Los enfoques alternativos posibles son

- @Query
 - Permite escribir la consulta manualmente
 - Flexible y claro
- Specification (Criteria API)
 - Para consultas dinámicas

```
Specification<Purchase> spec = (root, query, cb) -> {
    return cb.equal(root.get("user").get("username"), username);
}
```

- Permite combinar filtros dinámicamente
- QueryDSL
 - Alternativa más potente y tipada al Criteria API
 - Ideal para queries complejas y mantenibles
- Named Queries
 - Queries definidas de la entidad
 - Util para consultas reutilizables

2.3 Consultas con @Query

La anotación @Query permite escribir consultas directamente sobre el método del repositorio. Antes de utilizarla es importante entender la relación entre los lenguajes de consulta involucrados.

19. En la Práctica 1 se utilizaron consultas HQL (Hibernate Query Language). ¿Que diferencia hay entre HQL y JPQL (Java Persistence Query Language)? ¿Son intercambiables? ¿Cual de los dos acepta @Query por defecto?

HQL es el lenguaje de consultas propio de hibernate, mientras que JPQL es el estándar definido por JPA

Ambos trabajan sobre entidades, atributos y relaciones del modelo y no directamente sobre tablas y columnas SQL

La diferencia principal es que JPQL es el estándar JPA y HQL es la implementación de hibernate

Generalmente son intercambiables, aunque HQL puede soportar características adicionales específicas de Hibernate

La anotación @Query utiliza JPQL por defecto

20. ¿Que diferencia hay entre una consulta @Query con JPQL y una con nativeQuery = true? ¿Cuándo conviene cada una? Dar un ejemplo concreto del modelo para cada caso.

Una consulta @Query normal utiliza JPQL mientras que @Query(nativeQuery = true) utiliza SQL nativo

Con JPQL

- Se trabaja con entidades y atributos java
- Hibernate genera el SQL automáticamente
- Es portable entre motores de base de datos

```
@Query("""
    select p.user
    from Purchase p
    group by p.user
    having sum(p.totalPrice) >= :mount
""")
List<User> getUserSpendingMoreThan(float mount);
```

Con nativeQuery = true

- Se escribe SQL real
- Se usan tablas y columnas de la db

- Depende del motor utilizado

```
@Query(value = """
    SELECT *
    FROM routes r
    ORDER BY (
        SELECT AVG(rv.rating)
        FROM reviews rv
        JOIN purchases p ON rv.purchase_id = p.id
        WHERE p.route_id = r.id
    ) DESC
    LIMIT 3
""", nativeQuery = true)
List<Route> getTop3Routes();
```

Conviene usar

- JQPL para consultas orientadas al modelo y portables
 - SQL nativo cuando se necesitan funciones específicas del motor, optimizaciones particulares o características que JPQL no soporta fácilmente
21. ¿Cómo se pasan parámetros a una consulta @Query? Describir y comparar las dos formas: parámetros posicionales (?1, ?2) y parámetros nombrados (@Param). ¿Cuál es la forma recomendada y por qué?

Los parámetros en @Query pueden pasarse de dos formas

Parámetros posicionales

Se referencian por posición usando ?1, ?2, etc

Ejemplo

```
@Query("""
    select p
    from Purchase p
    where p.date between ?1 and ?2
""")
List<Purchase> getPurchasesBetween(Date start, Date end);
```

Donde cada posición referencia al parámetro de esa posición

Parámetros nombrados

Se usan nombres mediante :nombre y @Param

Ejemplo

```
@Query("""
    select p
```

```

    from Purchase p
    where p.date between :start and :end
    """)
List<Purchase> getPurchasesBetween(
    @Param("start") Date start,
    @Param("end") Date end
);

```

La forma recomendada es usar parámetros nombrados porque mejora la legibilidad y evita errores cuando se cambia el orden de parámetros. Además, la consulta queda más clara y facilita el mantenimiento

2.4 Paginación y ordenamiento

Spring Data JPA incorpora soporte nativo para paginación y ordenamiento a través de Pageable y Sort, sin necesidad de modificar las consultas manualmente. Esta funcionalidad es necesaria para algunas de las consultas que se implementaran en la sección siguiente.

22. ¿Qué es la interfaz Pageable? ¿Cómo se construye una instancia de Pageable? ¿Qué información encapsula (número de página, tamaño, ordenamiento)?

Pageable es una interfaz de Spring Data JPA que representa información de paginación y ordenamiento para una consulta. Permite indicar: número de página, cantidad de elementos por página, ordenamiento. Se construye normalmente usando PageRequest

Ejemplo

```

Pageable pageable = PageRequest.of(0, 10);

```

Esto representa la página 0 y 10 elementos por página

También puede tener ordenamiento

```

Pageable pageable = PageRequest.of(
    0,
    10,
    Sort.by("date").descending()
);

```

Y luego se usa como parámetro

```

Page<Purchase> findByUserUsername(String username, Pageable pageable);

```

23. ¿Que diferencia hay entre los tipos de retorno Page<T> y Slice<T>? ¿Cuándo conviene cada uno? ¿Qué consulta adicional ejecuta Page<T> que Slice<T> no ejecuta?

Page<T> y Slice<T> representan resultados paginados pero tienen una diferencia importante

- Page devuelve información completa de paginación
- Slice devuelve solo una porción de resultados

Page incluye

- contenido
- número de página
- tamaño
- total de páginas
- total de elementos
- información sobre página siguiente y anterior

Slice incluye

- contenido
- información sobre si existe una siguiente página
- no conoce el total de elementos ni el total de páginas

Page ejecuta una consulta adicional COUNT que Slice no

Conviene utilizar Page cuando se necesita mostrar cantidad total de páginas o resultados y Slice cuando sólo interesa navegar secuencialmente sin conocer el total

24. Mostrar como se invocará desde la capa de servicio un método paginado para obtener la segunda página de compras de un usuario, con 10 resultados ordenados por fecha descendente.

Para obtener la segunda página de compras de un usuario con 10 resultados ordenados por fecha descendente, desde la capa de servicio se usaría

```
Pageable pageable = PageRequest.of(
    1,
    10,
    Sort.by("date").descending()
);

Page<Purchase> purchases =
    purchaseRepository.findByUserUsername(username, pageable);
```

25. ¿Cómo se agrega ordenamiento a un Query Method sin usar Pageable? ¿Qué palabra clave se usa en el nombre del método? Mostrar un ejemplo concreto con el modelo.

Se agrega ordenamiento a un Query Method usando OrderBy en el nombre del método

```
List<Route> findByPriceLessThanOrderByNameAsc(float price);
```

2.5 Implementación de las consultas del modelo

Las siguientes consultas fueron implementadas en la Práctica 1 usando HQL con Session de Hibernate. El objetivo es reimplementarlas ahora usando repositorios Spring Data JPA, eligiendo para cada una la estrategia más adecuada: Query Method o @Query con JPQL.

26. Para cada consulta, indicar la estrategia elegida (Query Method o @Query) e implementarla en el repositorio correspondiente:

Consulta	Repositorio	Estrategia elegida
getAllPurchasesOfUsername(String username)	PurchaseRepository	Query Method
getUserSpendingMoreThan(float amount)	UserRepository	@Query
getTopNSuppliersInPurchases(int n)	SupplierRepository	@Query
getCountOfPurchasesBetweenDates(Date from, Date to)	PurchaseRepository	@Query
getRoutesWithStop(Stop stop)	RouteRepository	Query Method
getMaxStopOfRoutes()	RouteRepository	@Query
getRoutesNotSell()	RouteRepository	@Query
getTop3RoutesWithMaxRating()	RouteRepository	@Query
getMostDemandedService()	ServiceRepository	@Query
getTourGuidesWithRating1()	UserRepository	@Query

27. Implementar cada una de las consultas de la tabla anterior en el repositorio correspondiente, utilizando la estrategia indicada. Para las consultas que requieran paginación, utilizar Pageable como parámetro adicional.
28. Comparar la implementación de al menos dos consultas complejas (por ejemplo getTopNSuppliersInPurchases y getTop3RoutesWithMaxRating) entre la Práctica 1 y esta práctica. ¿Qué diferencias hay en términos de cantidad de código, legibilidad y acoplamiento a la infraestructura de persistencia? A partir de esta comparación, ¿qué es el código boilerplate y qué ventaja concreta representa su reducción para el mantenimiento de un proyecto real?

```
// Práctica 1
public List<Route> getTop3RoutesWithMaxRating() {
    return getSession()
        .createQuery("""
            select p.route
            from Review r
            join r.purchase p
            group by p.route
            order by avg(r.rating) desc
            """, Route.class)
```

```

        .setMaxResults(3)
        .getResultList();
    }

// Práctica 2
@Query("""
    select p.route
    from Review r
    join r.purchase p
    group by p.route
    order by avg(r.rating) desc
""")
List<Route> getTop3RoutesWithMaxRating(Pageable pageable);

```

La diferencia principal es que en la práctica 1 el repositorio estaba más acoplado a Hibernate directo, dependía de SessionFactory y Session

En Spring Data JPA, el repositorio sólo declara qué consulta necesita y Spring genera la implementación

Reduce código repetitivo

La ventaja es que el código queda más corto, legible y fácil de mantener

Sección 3 - Transacciones

En la Práctica 1 se implementó el manejo de transacciones utilizando `@Transactional` sobre la capa de servicio. En esta sección se profundiza sobre algunos aspectos de la gestión declarativa de transacciones en Spring que no se abordaron anteriormente, y se adapta el servicio a la nueva implementación.

3.1 Aspectos avanzados de `@Transactional`

29. ¿Cuál es el atributo `readOnly = true` en `@Transactional`? ¿Qué optimizaciones activa?
 ¿En qué métodos del servicio conviene aplicarlo?

El atributo `readOnly = true` en `@Transactional` indica que el método sólo realiza lectura y no debería modificar la base de datos

Ejemplo:

```

@Transactional(readOnly = true)
public Optional<User> getUserById(Long id) {
    return userRepository.findById(id);
}

```

Puede activar optimizaciones como evitar chequeos innecesarios de cambios sobre entidades, reducir trabajo del contexto de persistencia y mejorar la intención del código

Conviene aplicarlo en métodos de consulta que sólo obtienen datos

30. ¿Cómo maneja Spring el rollback automático? ¿Sobre qué tipos de excepción hace rollback por defecto? ¿Cómo se configura para el rollback? Dar un ejemplo con el modelo.

Spring maneja el rollback automático cuando ocurre una excepción durante la ejecución de un método anotado con `@Transactional`

Por defecto, Spring hace rollback únicamente sobre excepciones no verificadas (`RuntimeException` y `Error`)

Las excepciones verificadas (`Exception`, `IOException`, etc.) no provocan rollback automáticamente

Para configurar rollback sobre otras excepciones se usa

```
@Transactional(rollbackFor = Exception.class)
```

Ejemplo

```
@Transactional(rollbackFor = ToursException.class)
public void assignDriverByUsername(String username, Long idRoute)
    throws ToursException {

    DriverUser driver = userRepository
        .findDriverUserByUsername(username)
        .orElseThrow(() ->
            new ToursException("No existe el chofer"));

    Route route = routeRepository.findById(idRoute)
        .orElseThrow(() ->
            new ToursException("No existe la ruta"));

    route.addDriver(driver);

    routeRepository.save(route);
}
```

Si ocurre un `ToursException`, Spring hará rollback y deshacerá cualquier modificación realizada durante la transacción

3.2 Adaptación de la capa de servicio

La capa de servicio implementada en la Práctica 1 ya utiliza `@Transactional`. Sin embargo, al migrar los repositorios a Spring Data JPA, las llamadas internas al servicio deben actualizarse para usar los nuevos repositorios en lugar de la Session de Hibernate.

31. Revisar y actualizar la capa de servicio de la Práctica 1 para que utilice los repositorios Spring Data JPA. Reemplazar todas las llamadas directas a la Session (`session.save`, `session.get`, `session.createQuery`, etc.) por las operaciones equivalentes de los repositorios. La lógica de negocio y las anotaciones `@Transactional` ya existentes no deben modificarse salvo que sea necesario por los cambios anteriores.

Sección 4 - DTOs y Proyecciones

Hasta ahora todos los repositorios devuelven entidades completas. En muchos casos de uso real, la aplicación solo necesita un subconjunto de los datos. Para esto Spring Data JPA ofrece proyecciones y DTOs.

4.1 Qué es un DTO y cuando usarlo

32. ¿Qué es un DTO (Data Transfer Object)? ¿Cuál es su propósito dentro de la arquitectura de una aplicación? ¿Por qué no siempre conviene devolver una entidad JPA directamente desde la capa de acceso a datos?

Un DTO (Data Transfer Object) es un objeto utilizado para transferir datos entre capas de una aplicación, generalmente conteniendo solo la información necesaria para un caso de uso específico

Su propósito es

- reducir la cantidad de datos transferidos
- desacoplar la capa de persistencia de otras capas
- evita exponer directamente las entidades JPA
- mejorar seguridad y mantenimiento

No siempre conviene devolver entidades JPA directamente porque

- pueden contener relaciones innecesarias
- pueden producir cargas LAZY inesperadas
- pueden generar ciclos de serialización
- acoplan la estructura interna de persistencia con el exterior
- exponen datos que no deberían mostrarse

33. Describir dos situaciones concretas del modelo donde devolver un DTO sería más adecuado que devolver la entidad completa. Para cada caso indicar qué campos contendría el DTO y por qué.

Caso 1: Resumen de rutas

En lugar de devolver la entidad completa `Route`, conviene devolver un DTO con información resumida para listados

```
public class RouteSummaryDTO {
    private String routeName;
    private float price;
    private int stopsCount;
}
```

Esto evita enviar relaciones completas, choferes, guías, compras, reviews, etc.

Caso 2: Información resumida de compras

Para mostrar historial de compras de un usuario

```
public class PurchaseDTO {
    private String code;
    private Date date;
    private String routeName;
    private float totalPrice;
}
```

No hace falta devolver todos los ItemService, reviews completas, usuario completo, relaciones asociadas

En ambos casos, el DTO reduce datos transferidos, evita problemas de serialización y desacopla la capa de persistencia de la presentación

34. ¿Qué riesgos concretos tiene exponer entidades JPA directamente como respuesta de un servicio o endpoint? Mencionar al menos: ciclos de serialización y acoplamiento entre capas.

Exponer entidades JPA directamente desde un servicio o endpoint tiene varios riesgos

Uno de los principales es la aparición de ciclos de serialización en relaciones bidireccionales

Ejemplo

```
Purchase -> User
User -> purchases
```

Al serializar una compra, también se serializa el user, que a su vez serializa nuevamente sus compras, produciendo recursión infinita o errores

Otro problema es el fuerte acoplamiento entre capas.

Si se exponen entidades directamente, cualquier cambio en el modelo de persistencia afecta automáticamente a las respuestas del sistema y a las capas consumidoras

Además

- pueden exponerse atributos sensibles

- pueden dispararse cargas lazy inesperadas
- se transfieren más datos de los necesarios

4.2 Implementación

35. Definir e implementar un DTO que resuma información de las rutas del modelo: nombre de la ruta, cantidad de compras realizadas y el precio promedio de esas compras. Implementar la consulta correspondiente en RouteRepository.

Sección 5 - Borrado Lógico (Soft Delete)

El borrado lógico es una estrategia de persistencia que evita eliminar físicamente registros de la base de datos, marcándolos en cambio como inactivos. Spring Data JPA y Hibernate ofrecen mecanismos para implementarlo de forma transparente para el resto de la aplicación.

5.1 Concepto y estrategias

36. ¿Que es el borrado lógico (soft delete)? Pensar en el modelo de tours: si un usuario decide darse de baja del sistema, ¿tiene sentido eliminar físicamente su registro? ¿Qué ocurriría con todas las compras que ese usuario ha realizado? Describir por qué el borrado lógico resuelve este problema y qué ventaja ofrece frente al borrado físico en este caso concreto.

El borrado lógico consiste en no eliminar físicamente un registro de la base de datos, sino marcarlo como inactivo o eliminado

En el modelo de tours, si un usuario se da de baja, no conviene eliminarlo físicamente porque ese usuario puede tener compras asociadas. Si se borra el registro real, podrían quedar inconsistencias o perderse datos históricos de compras, reviews y facturación. Con soft delete, el usuario queda marcado como inactivo por un campo "active" o a través del seteo de una fecha en un campo "deletedAt"

De esta forma se conserva el historial de compras y no se nullean relaciones con Purchase. Además, el usuario dejaría de aparecer en consultas normales pero se sigue pudiendo recuperar su información

La ventaja del borrado lógico es que mantiene la integridad histórica del sistema sin perder datos importantes

37. Describir dos estrategias para implementar soft delete en JPA/Hibernate: campo booleano (active/deleted) y campo de fecha (deletedAt). ¿Qué diferencias hay entre ambas en cuanto a consultas y recuperación de datos?

Campo booleano

- Se agrega un atributo como "active"
- Al eliminar una entidad, cambia ese valor a false
- Es simple, fácil de consultar
- No guarda cuándo ocurrió el borrado

Campo fecha

- Se agrega un atributo como "deletedAt"
- Al eliminar una entidad, ese atributo toma un valor no nulo
- Permite saber cuándo se borró aunque ocupa más espacio y las consultas son un poco más complejas

38. ¿Que es la anotación @SQLDelete de JPA? ¿Qué permite hacer? ¿Cómo se combina con @Where para que las consultas ordinarias ignoren los registros borrados? Mostrar un ejemplo de uso sobre la entidad User.

@SQLDelete permite redefinir qué SQL ejecuta Hibernate cuando se llama a delete() sobre una entidad

En lugar de hacer un DELETE físico, puede ejecutarse con UPDATE para marcar el registro como inactivo

@Where agrega automáticamente una condición a todas las consultas generadas por Hibernate para esa entidad

Ejemplo sobre User

```
package unlp.info.bd2.model;

import jakarta.persistence.*;
import org.hibernate.annotations.SQLDelete;
import org.hibernate.annotations.Where;

@Entity
@Table(name = "users")
@SQLDelete(sql = """
    UPDATE users
    SET active = false
    WHERE id = ?
""")
@Where(clause = "active = true")
public class User {
    @Id
    @GeneratedValue
    private Long id;

    private boolean active = true;
}
```

Cuando se ejecuta el método delete, hibernate ejecuta un UPDATE sobre el atributo active en lugar de ejecutar un DELETE. Además, gracias al @Where, todas las consultas normales ignorarán usuarios inactivos

5.2 Implementación en el modelo

39. Implementar soft delete para la entidad User usando @SQLDelete y @Where. La implementación debe:

- g) Agregar el campo necesario a la entidad User (campo y anotaciones).
- h) Redefinir el comportamiento de delete para que marque el registro en lugar de eliminarlo.
- i) Asegurar que todas las consultas ordinarias (findAll, findById, Query Methods) excluyan automáticamente los usuarios borrados.

```
@Entity
@Table(name="users")
@SQLDelete(sql = """
    UPDATE users
    SET active = false
    WHERE id = ?
""")
@Where(clause = "active = true")
@Inheritance(strategy = InheritanceType.JOINED)
public class User {
    ...

    @Column(nullable=false)
    private boolean active;

    ...
}
```

40. La implementación actual muestra el método canBeDeactivate() en User, DriverUser y TourGuideUser. El objetivo de estos métodos es evitar que se eliminen usuarios que poseen compras o asignaciones y, por tanto queden inconsistencias en la base de datos. Describa como esta nueva implementación afecta dichos métodos.

Atención: La anotación @Where de JPA se aplica globalmente a todas las consultas que Hibernate genera para esa entidad. Si se necesita consultar usuarios inactivos (por ejemplo, para un panel de administración), es necesario usar SQL nativo o JPQL con una consulta explícita que omita el filtro.

Con soft delete, ese método cambia de sentido

Antes servía para evitar borrar físicamente usuarios con compras porque eso podía romper relaciones

Ahora como delete no elimina el registro físico ya no se rompen las FK ni se pierden compras históricas

Este método puede seguir siendo útil para validar reglas de negocio antes de marcar al usuario

como inactivo, pero el soft delete reduce el problema de la integridad referencial así que no es completamente necesario